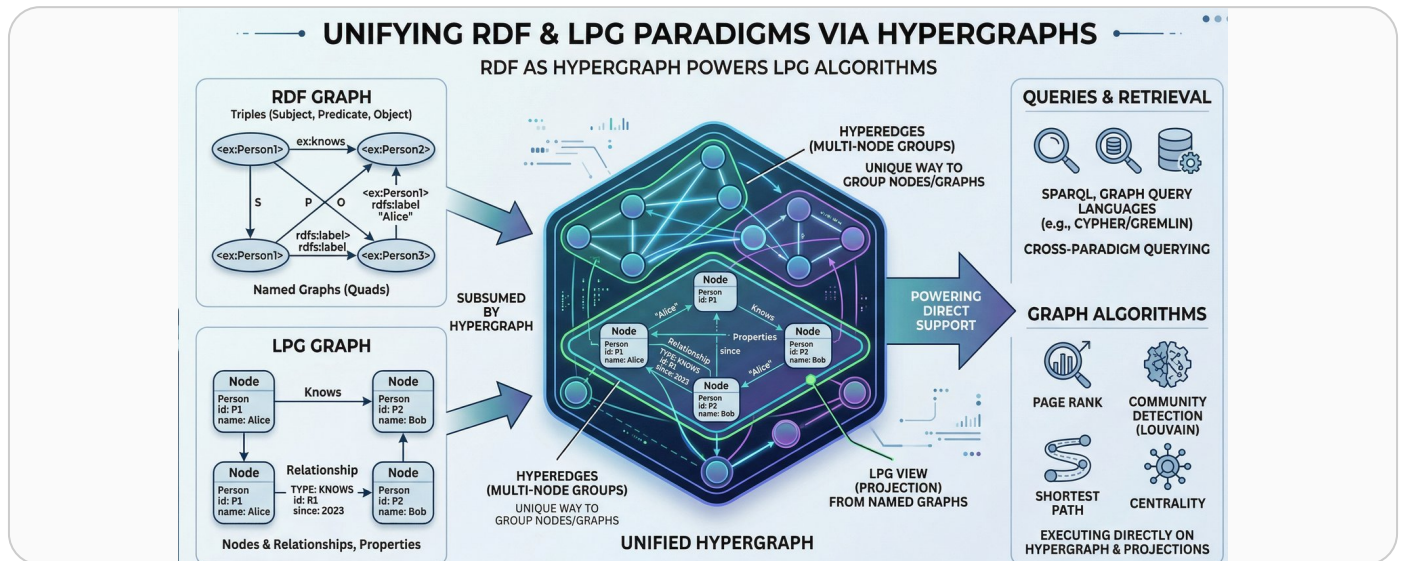


The RDF vs. LPG Debate Is Quickly Becoming Irrelevant



RDF is already a hypergraph system; the LPG is a projection inside it

Zachary Welz, PhD (Peerless Tech.)

2026-03-23

Previously discussed on [LinkedIn](#)

The graph community has spent years debating RDF versus Labeled Property Graphs (LPGs). I think both sides are finally discovering what's been right in front of us, and has become more relevant with the latest specs: RDF is already a hypergraph system; once we treat it that way, the distinction collapses.

A note on the examples: the predicates, classes, and graph structures used throughout this post are deliberately simplified for clarity. In a real system, the ontological complexity would be significantly higher; predicates would be drawn from established vocabularies (or carefully defined domain ontologies), class hierarchies would be deeper, and the relationships between subjects, properties, and named graphs would carry far more nuance than `ex:hasFur` and `ex:coLocatedWith` suggest. The point here is the architectural pattern, not the vocabulary.

The Simple Version of a Knowledge Graph

Most introductions to knowledge graphs start somewhere like:

```

1  @prefix ex: <http://example.org/> .
2  @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
3
4  ex:Dog rdfs:subClassOf ex:Animal ;
5      ex:hasCharacteristic ex:Furry ;
6      ex:hasLegCount 4 ;
7      ex:isSpecies ex:CanisLupusFamiliaris .

```

It's clean, readable, and useful, but everything we know about a "dog" is flattened into a single subject. The concept of dog is the node, and everything hangs off it. In this "ontological world" a dog cannot exist as a `ex:Dog` until it has been established within the ontology. This works until it doesn't (ignore the tautology).

What If a Concept Is the Graph?

Named graphs let us do something more powerful. Instead of describing a dog as a node with properties, we describe the *observation*; a named graph whose contents establish what it represents:

```

1  @prefix ex: <http://example.org/> .
2  @prefix obs: <http://example.org/observations/> .
3
4  obs:animal-obs-2024-0317 {
5      obs:animal-obs-2024-0317 ex:legCount 4 ;
6      ex:hasFur true ;
7      ex:observedMass "12.5kg" ;
8      ex:observedLocation ex:CentralPark ;
9      ex:observedDate "2024-03-17"^^xsd:date .
10 }

```

Notice what's *not* in this graph: there's no assertion that this is a dog. The triples inside the named graph are epistemically grounded; someone observed an animal with four legs, fur, and a mass of 12.5kg. That's what was actually seen, and it doesn't require a domain ontology to capture the *implications* (or assumptions, or conclusion, ...) of the observation.

The *classification* itself might live elsewhere:

```

1  @prefix ex: <http://example.org/> .
2  @prefix obs: <http://example.org/observations/> .
3  @prefix classify: <http://example.org/classification/> .
4
5  classify:species-assignments {
6      obs:animal-obs-2024-0317 a ex:Dog .
7      obs:animal-obs-2024-0317 ex:isSpecies ex:CanisLupusFamiliaris .
8  }

```

This is a (simple) hypergraph in practice. The named graph `obs:animal-obs-2024-0317` is both a container of triples and a node that participates in triples elsewhere. Assertions are separated from interpretation. Provenance is separated from both. The *intent* of data captured within a named graph is decoupled from its *storage*.

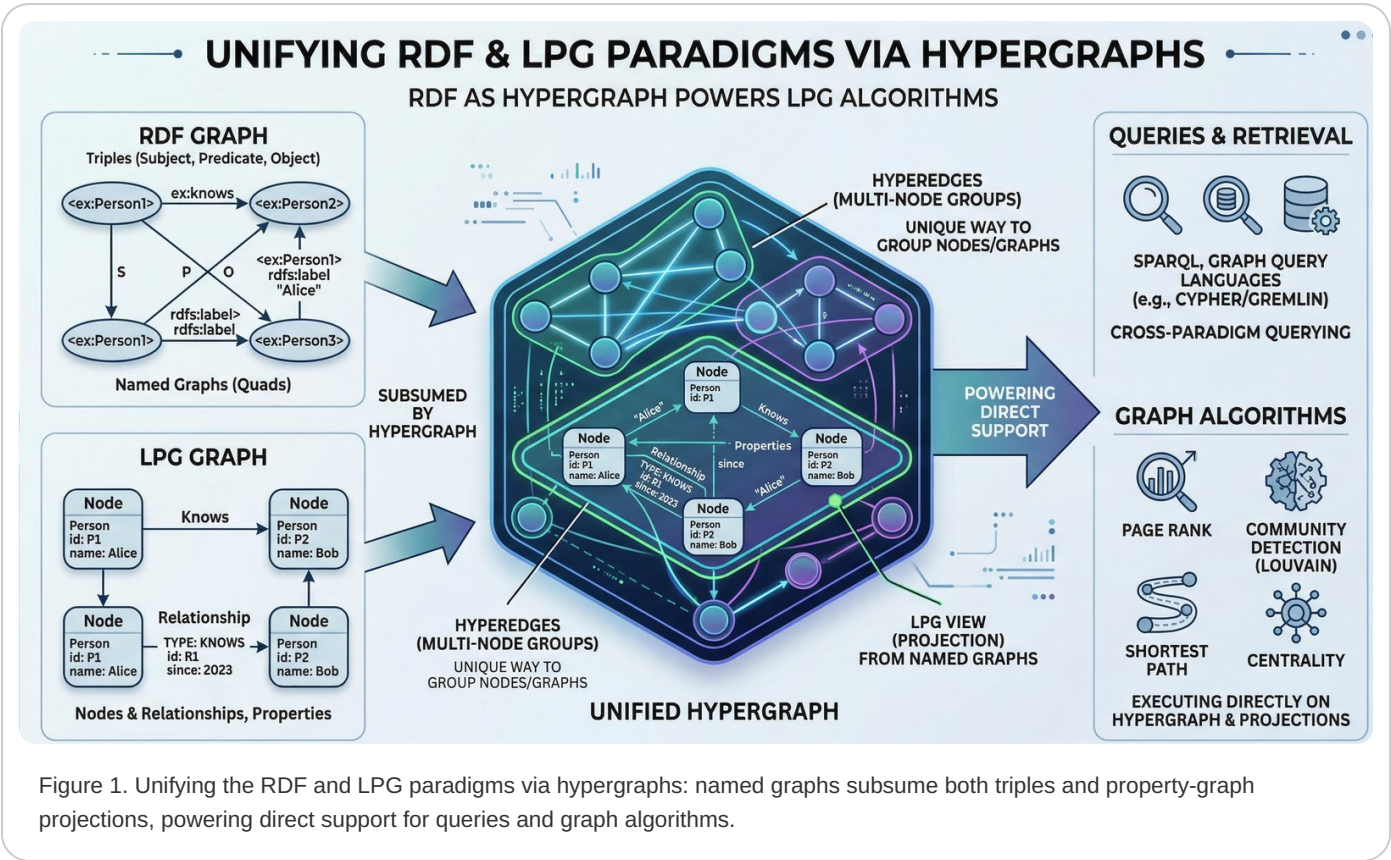
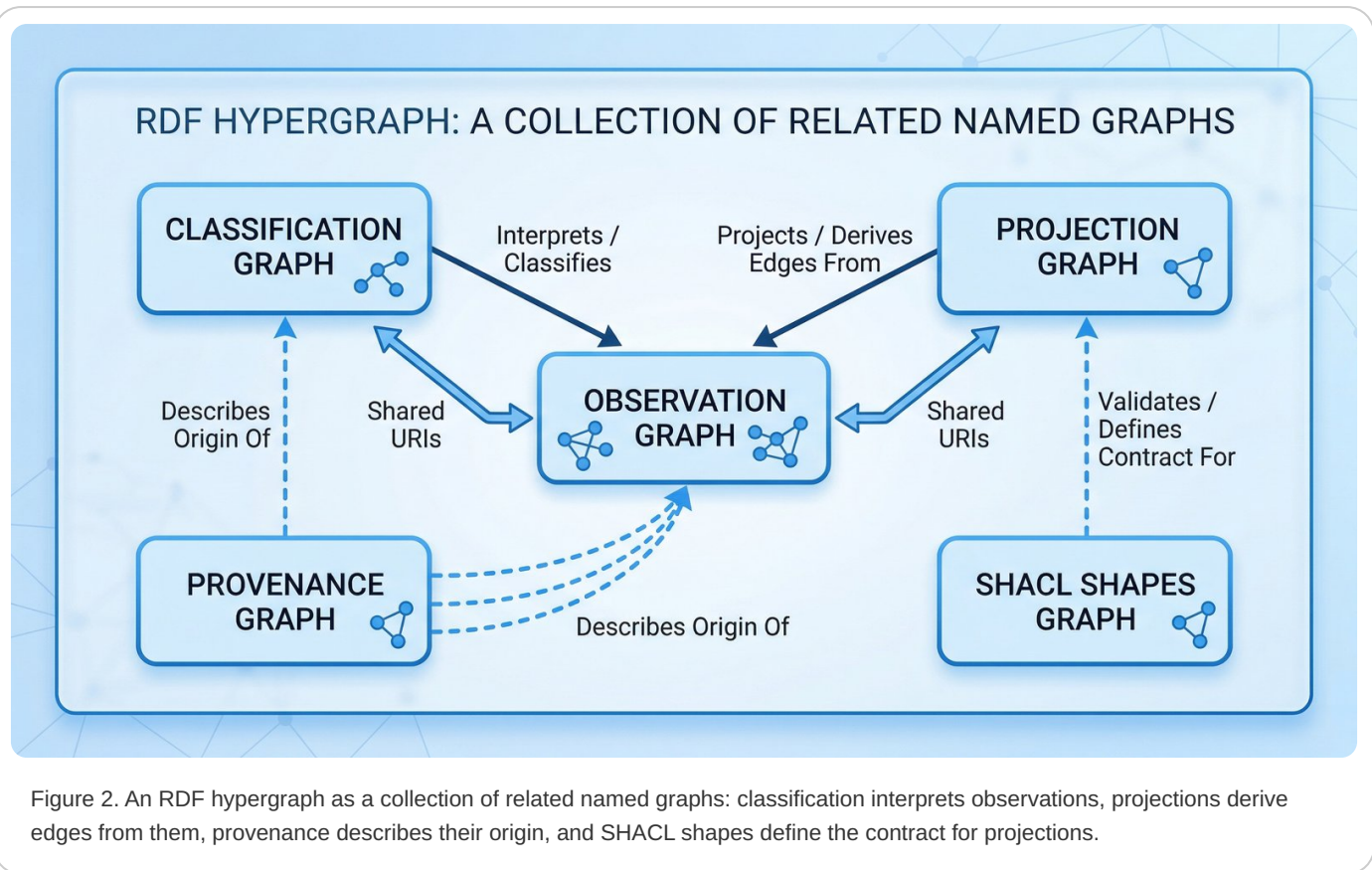


Figure 1. Unifying the RDF and LPG paradigms via hypergraphs: named graphs subsume both triples and property-graph projections, powering direct support for queries and graph algorithms.

The Hypergraph Structure

Here's how these named graphs relate to each other within the containing hypergraph. Each named graph serves a distinct role, e.g., as observations, classifications, projections, and constraints, but they reference each other through shared URIs, forming a self-organizing system:



The key insight is that every named graph is simultaneously a container (holding its own triples) and a participant (referenced by URI in other graphs). Classification references observations. Projections reference observations. SHACL shapes validate projections. No graph owns another; instead, they interrelate through URIs within a single hypergraph.

Note: the particulars of the hypergraph structure here are only to serve as an example. I think best practices and tooling to support this are where research is moving.

Why This Matters for LPG Users

The LPG world (Neo4j, NetworkX, and similar tools) exists primarily because people need to run graph algorithms: centrality, community detection, pathfinding. These algorithms require a projected, simplified structure: nodes, edges, and properties on both.

I am starting to see that the projected LPG structure can itself be managed as another named graph within the same hypergraph.

```
1  @prefix ex: <http://example.org/> .
2  @prefix proj: <http://example.org/projections/> .
3  @prefix obs: <http://example.org/observations/> .
4
5  proj:animal-network-v1 {
6    obs:animal-obs-2024-0317 ex:colocatedWith obs:animal-obs-2024-0318 ;
7    proj:weight 0.85 ;
8    proj:algorithm "spatial-proximity" .
9  }
```

This projection graph captures exactly the structure a graph analytics engine needs (e.g., co-location edges with weights) while remaining part of the same RDF hypergraph system. You project it into Neo4j or NetworkX when you need to compute, but the source of truth stays in one place. No managing two separate graph systems with fragile synchronization. And the underlying distinction between LPG and RDF in this case fits perfectly into this proposed "self-organizing hypergraph" framework.

SHACL: Making the Hypergraph Self-Describing

How do we make this framework practical? If the projection graph is going to be handed off to a graph analytics engine, something needs to define what a valid projection edge looks like. SHACL shapes, stored in their own named graph, do exactly that. How do we support RDF-based knowledge graphs as a transfer layer framework? Similarly, we use SHACL (stored in named graphs that represent, capture, depict the information "handshake") to negotiate the effective "data contract", and even the self-describing nature of information on either side.


```

1  @prefix sh: <http://www.w3.org/ns/shacl#> .
2  @prefix ex: <http://example.org/> .
3  @prefix proj: <http://example.org/projections/> .
4  @prefix shapes: <http://example.org/shapes/> .
5  @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
6
7  shapes:projection-constraints {
8      ex:ProjectionEdgeShape a sh:NodeShape ;
9      sh:targetSubjectsOf ex:colocatedWith ;
10     sh:property [
11         sh:path ex:colocatedWith ;
12         sh:nodeKind sh:IRI ;
13     ] ;
14     sh:property [
15         sh:path proj:weight ;
16         sh:datatype xsd:decimal ;
17         sh:minInclusive 0.0 ;
18         sh:maxInclusive 1.0 ;
19         sh:minCount 1 ;
20     ] ;
21     sh:property [
22         sh:path proj:algorithm ;
23         sh:datatype xsd:string ;
24         sh:minCount 1 ;
25     ] .
26 }

```

This simple shape says: every node that has a `colocatedWith` edge must point to another IRI (not a literal), must carry a `proj:weight` between 0 and 1, and must declare which algorithm produced it. Before you project data into Neo4j, you validate `proj:animal-network-v1` against `shapes:projection-constraints`. The data handshake between the RDF hypergraph and the downstream analytics tool is defined within the hypergraph itself. The complexity of these definitions will grow with expressivity, but the general approach is the same.

This is the part that matters most for bridging the RDF--LPG gap. The SHACL graph doesn't just constrain data, but also documents the contract that the i.e. projection graph must satisfy for the consuming system. A NetworkX pipeline can trust that every edge it receives has a numeric weight and a named algorithm, because the shape enforced it at the source. Different downstream tools can have different shapes graphs, all coexisting in the same hypergraph, each defining what their projection needs to look like.

What's Actually Holding Us Back

The concept is sound. The RDF specifications, especially with RDF 1.2 and RDF-star, are evolving toward first-class hypergraph support. The real gap is tooling and methodology.

We lack a repeatable, flexible, and simple process for generating and managing hypergraphs. In my experience, it has been a genuine challenge over the past few years to effectively manage named graphs organized in any way more complex than "graph identifier equals subject identifier." That's the simplest case, and we're already struggling with it.

The value of URIs automatically resolving in RDF is that you can manage context, instance-level variation, and provenance across many different places, all independently. But we keep building tools that often assume a flat triple store with a single default graph, and then wonder why RDF feels harder (or has a higher investment penalty) than Neo4j.

Where This Is Going

The RDF versus LPG debate will soon be subsumed by the recognition that a well-managed RDF hypergraph contains the LPG as a projection. Observations, classifications, SHACL constraints, and algorithm-ready structures all coexist as named graphs within the same system. The shapes graph defines the handshake. The projection graph satisfies it. The analytics engine consumes it.

We as the community need to stop debating format and start building the hypergraph management layer/tools that makes this practical. The spec is nearly there, but implementations need to catch up.

Zachary Welz, PhD



© 2026 Zachary Welz.

CC BY 4.0